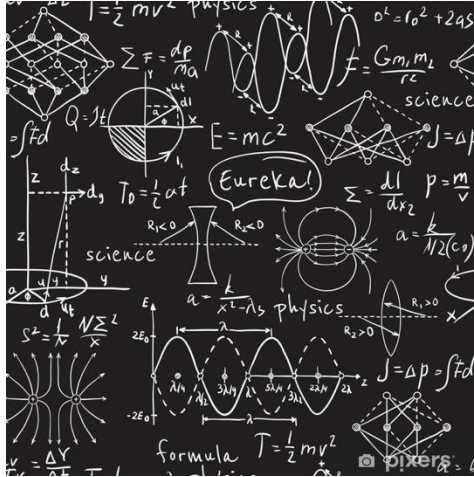


Computational Physics

Lecture 1: Introduction, Syllabus, Technical Details

Theory – Experiment Classic Duo

Theory (model)



predicts



validate

Experiments (reality)



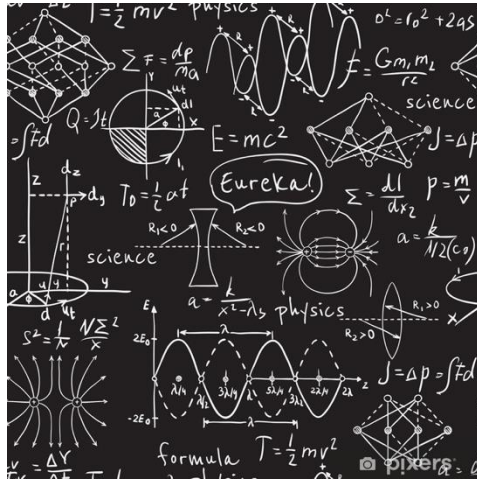
Before the creation of the first computer in 1940s, this was the norm.

The physics you learned typically focuses on fundamental theories with example whose solutions can always be written with pen and paper.

Why this picture do not represent the real world?

Theory – Experiment Classic Duo

Theory (model)



predicts



← validate



Experiments (reality)



Several problems:

- 1) Theoretical research is limited by the possibility of finding analytical solutions, excluding complex equations
- 2) Experiments might be too difficult to realize or very expensive or we just want an analytical insight

Computational Physics

Remarkably rare that any new physics question can be answered solely by analytical methods

Computer calculations can provide:

1.Process and analyse large amounts of data from measurements; fit to theoretical models; display and animate graphically

Ex: search for "events" in particle physics, image analysis in astronomy.

2.Numerical solution of equations that cannot be accomplished by analytical techniques (coupled, nonlinear etc.)

Ex: fluid dynamics (Navier Stokes), numerical relativity (Einstein's field equations), electronic ground state wavefunctions in solid state systems, nonlinear growth equations

3.Computer "experiments": simulate physical phenomena, observe and extract quantities as in experiments, explore simplified model systems for which no solution is known.

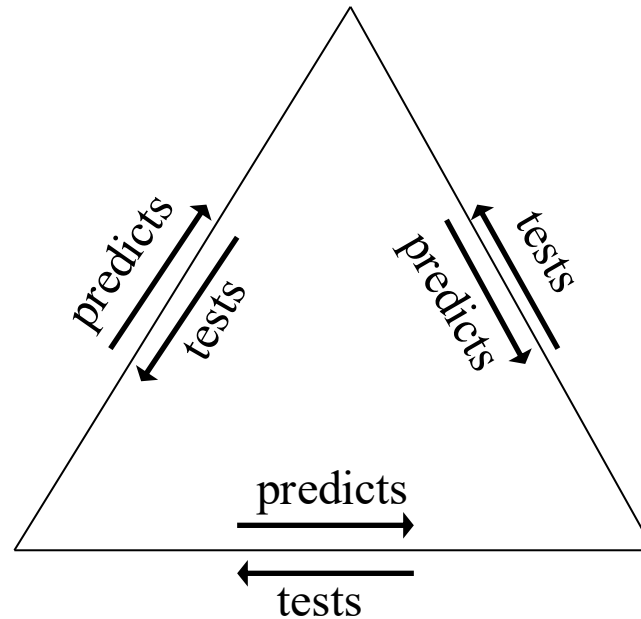
Ex: molecular simulations of materials, protein folding, planetary dynamics (N-body dynamics).

Theory – Experiment – Computation Trio

**Computational physics is
the “third way” of physics!**

Computational Physics

Performs idealized
"experiments" on the
computer, solves physical
models numerically



Theoretical Physics

Construction and
mathematical (analytical)
analysis of idealized
models and hypotheses
to describe nature

Experimental Physics

Quantitative measurement
of physical phenomena

Timeline of Scientific Computing (Wikipedia)

Before modern computers

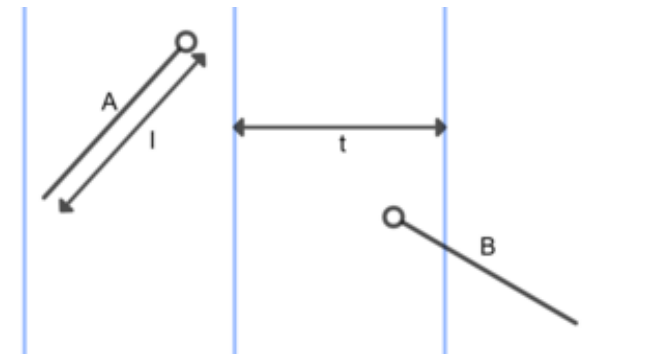
18th century [\[edit \]](#)

- Simpson rediscovers [Simpson's rule](#), a century after [Johannes Kepler](#) (who derived it in 1615 after seeing it used for wine barrels).
- 1733 – The French naturalist Comte de Buffon poses [his needle problem](#).^{[1][2]}
- Euler comes up with [a simple numerical method](#) for integrands.^{[3][4][5]}

method to perform integrals numerically

inspiration for Monte Carlo methods, estimation of π from probability needle lies across strips

$$p = \frac{2}{\pi} \cdot \frac{l}{t}$$



method to solving ordinary differential equations

Timeline of Scientific Computing (Wikipedia)

Before modern computers

19th century [\[edit \]](#)

- First formulation of [Gram-Schmidt orthogonalisation](#) by Laplace,^[9] to be further improved decades later.^{[10][11][12][13]}
- Babbage in 1822, began work on a machine made to compute/calculate values of polynomial functions automatically by using the method of finite differences. This was eventually called the [Difference engine](#).
- Lovelace's note G on the [Analytical Engine](#) (1842) describes an algorithm for generating [Bernoulli numbers](#). It is considered the first algorithm ever specifically tailored for implementation on a computer, and thus the first-ever computer programme.^{[14][15]} The engine was never completed, however, so her code was never tested.^[16]
- [Adams-Bashforth](#) method published.^[17]
- In applied mathematics, Jacobi develops [technique for solving numerical equations](#).^{[18][19][20]}
- Gauss Seidel first published.
- To help with computing tides, [Harmonic Analyser](#) is built in 1886.
- 850 CE: Al-Kindi's frequency analysis – First systematic cryptanalysis technique for breaking substitution ciphers.^[21]
- 1206: Al-Jazari's programmable orchestra – Mechanical automata using pegged cylinders for sequence control (early program storage).^[22]
- 1676: Leibniz's chain rule – Foundation for calculus-based optimization later used in backpropagation.^[22]
- 1738/1763: Bernoulli's utility theory & Bayes' theorem – Probabilistic frameworks for decision-making algorithms.^[22]

Timeline of Scientific Computing (Wikipedia)

Before modern computers

1900s (decade) [\[edit \]](#)

- 1900 – [Runge](#)'s work followed by [Martin Kutta](#) to invent the [Runge-Kutta method](#) for approximating integration for differential equations.^{[23][24]}

1920s [\[edit \]](#)

better method to solving ordinary differential equations

- 1922 – [Lewis Fry Richardson](#) introduces [numerical weather forecasting](#) by manual calculation, using methods originally developed by [Vilhelm Bjerknes](#) as early as 1895.^[27]
^[28]
- 1926 – [Grete Hermann](#) publishes foundational paper for [computer algebra](#), which established the existence of [algorithms](#) (including complexity bounds) for many of the basic problems of [abstract algebra](#), such as [ideal](#) membership for [polynomial rings](#).^[29]
- 1926 Adams-Moulton method.

- 1927 – [Douglas Hartree](#) creates what is later known as the [Hartree–Fock method](#), the first [ab initio quantum chemistry methods](#). However, manual solutions of the Hartree–Fock equations for a medium-sized atom were laborious and small molecules required computational resources far beyond what was available before 1950.

method to approximate wave function ψ and energy of a quantum many-body system in a stationary state

Timeline of Scientific Computing (Wikipedia)

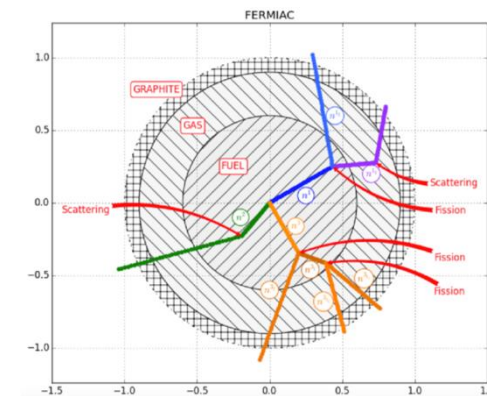
Advent of computers

1930s [[edit](#)]

This decade marks the first major strides to a modern computer, and hence the start of the modern era.

- Fermi's Rome physics research group (informal name *I ragazzi di Via Panisperna*) develop statistical algorithms based on [Comte de Buffon's](#) work, that would later become the foundation of the [Monte Carlo method](#). See also [FERMIAC](#).
- Shannon explains how to use electric circuits to do Boolean algebra in "[A Symbolic Analysis of Relay and Switching Circuits](#)"
- [John Vincent Atanasoff](#) and [Clifford Berry](#) create the first electronic non-programmable, digital computing device, the [Atanasoff–Berry Computer](#), from 1937-42.
- [Complex number calculator](#) created by Stibitz.
- At [Columbia University's](#) Rutherford Laboratory [Wallace J. Eckert](#) uses commercial tabulating machinery from [IBM](#), some of it specially modified, for scientific computation.

Analog computer and probabilistic methods to estimate neutron collisions, scattering and fission



Timeline of Scientific Computing (Wikipedia)

Advent of computers

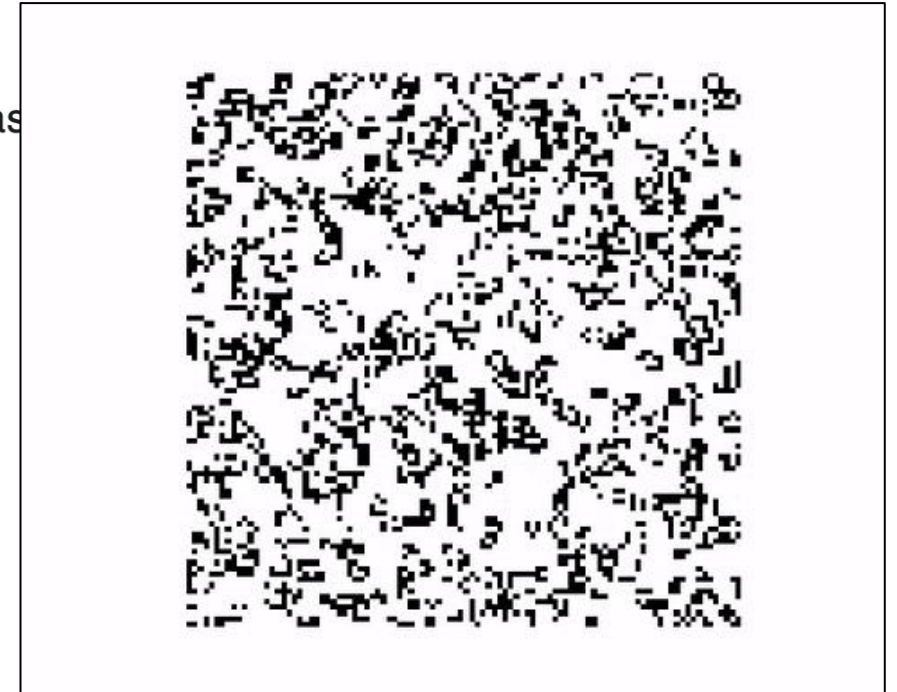
1940s [\[edit\]](#)

- 1947 – Metropolis algorithm for Monte Carlo simulation (named one of the top-10 [algorithms](#) of the 20th century)^[30] invented at Los Alamos by von Neumann, Ulam and Metropolis.^{[31][32][33]}
- [George Dantzig](#) introduces the simplex method (named one of the top 10 algorithms of the 20th century)^[30] in 1947.^[34]
- Ulam and von Neumann introduce the notion of cellular automata.^[35]
- Turing formulated the LU decomposition method.^[36]
- [A. W. H. Phillips](#) invents the [MONIAC hydraulic computer](#) at LSE, better known as
- First hydro simulations occurred at Los Alamos.^{[39][40]}

Monte Carlo is a code name for the project in the contest of nuclear weapon development

Metropolis algorithm is fundamental for sampling probability distributions in statistical mechanics

LU method solves linear equations

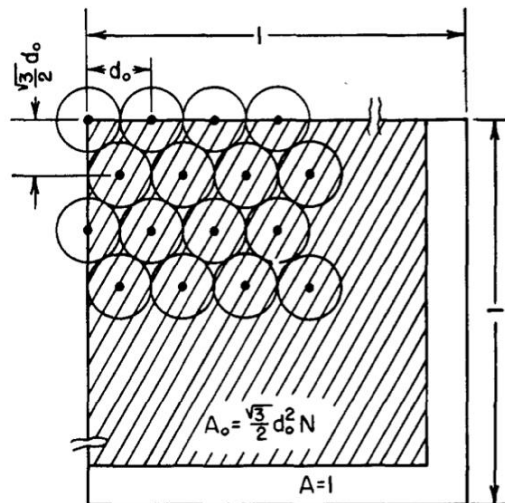


Timeline of Scientific Computing (Wikipedia)

1950s [[edit](#)]

- First successful weather predictions on a computer occurred.^{[41][42]}
- Hestenes, Stiefel, and Lanczos, all from the Institute for Numerical Analysis at the National Bureau of Standards, initiate the development of Krylov subspace iteration methods.^{[43][44][45][46]} Named one of the top 10 algorithms of the 20th century.^[30]
- Equations of State Calculations by Fast Computing Machines introduces the Metropolis–Hastings algorithm.^[47]
- Molecular dynamics invented by Bernie Alder and Wainwright^{[48][49]}
- A S Householder invents his eponymous matrices and transformation method (voted one of the top 10 algorithms of the 20th century).^[50]

First numerical simulations of particles



THE JOURNAL OF CHEMICAL PHYSICS

VOLUME 21, NUMBER 6

JUNE, 1953

Equation of State Calculations by Fast Computing Machines

NICHOLAS METROPOLIS, ARIANNA W. ROSENBLUTH, MARSHALL N. ROSENBLUTH, AND AUGUSTA H. TELLER,
Los Alamos Scientific Laboratory, Los Alamos, New Mexico

AND

EDWARD TELLER,* *Department of Physics, University of Chicago, Chicago, Illinois*

(Received March 6, 1953)

A general method, suitable for fast computing machines, for investigating such properties as equations of state for substances consisting of interacting individual molecules is described. The method consists of a modified Monte Carlo integration over configuration space. Results for the two-dimensional rigid-sphere system have been obtained on the Los Alamos MANIAC and are presented here. These results are compared to the free volume equation of state and to a four-term virial coefficient expansion.

Timeline of Scientific Computing (Wikipedia)

- 1953 – [Enrico Fermi](#), [John Pasta](#), [Stanislaw Ulam](#), and [Mary Tsingou](#) discover the [Fermi–Pasta–Ulam–Tsingou problem](#) through computer simulations of a vibrating string.^[51]
- A team led by [John Backus](#) develops the FORTRAN compiler and programming language at [IBM](#)'s research centre in [San Jose, California](#). This sped the adoption of scientific programming,^{[52][53][54]} and is one of the [oldest extant programming languages](#), as well as one of the most popular in science and engineering.

computer simulations of a vibrating string that included a non-linear term exhibited a very complicated quasi-periodic behavior



The original intent was to find a physics problem worthy of numerical simulation on the then-new [MANIAC](#) computer. Fermi felt that thermalization would pose such a challenge. As such, it represents one of the earliest uses of digital computers in mathematical research; simultaneously, the unexpected results launched the study of [nonlinear systems](#).

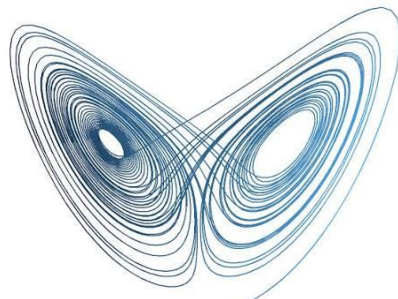
Timeline of Scientific Computing (Wikipedia)

1960s [[edit](#)]

How a continuum object deforms under stress

- 1960 – First recorded use of the term "[finite element method](#)" by [Ray Clough](#) to describe the earlier methods of [Richard Courant](#), [Alexander Hrennikoff](#) and [Olgiard Zienkiewicz](#) in structural analysis.^[55]
- 1961 – [John G.F. Francis](#)^{[56][57]} and [Vera Kublanovskaya](#)^[58] invent [QR factorization](#) (voted one of the top 10 algorithms of the 20th century).
- 1963 – [Edward Lorenz](#) discovers the [butterfly effect](#) on a computer, attracting interest in [chaos theory](#).^[59]
- 1961 – Using computational investigations of the [3-body problem](#), [Michael Minovitch](#) formulates the [gravity assist](#) method.^{[60][61]}
- 1964 – Molecular dynamics invented independently by [Aneesur Rahman](#).^[62]
- 1965 – [fast Fourier transform](#) developed by [James W. Cooley](#) and [John W. Tukey](#).^[63]
- 1964 – [Walter Kohn](#), with [Lu Jeu Sham](#) and [Pierre Hohenberg](#), instigates the development of [density functional theory](#),^{[64][65]} for which he shares the 1998 Nobel Chemistry Prize with [John Pople](#).^[66] This contribution is arguably the earliest work to which Nobels were given for a computer program or computational technique.
- First regression calculations in economics.

Sensitivity to initial conditions



method that estimate electron density in many-body systems

Physics and Computers

Some of the *hardest problems in physics* today are *computational problems*

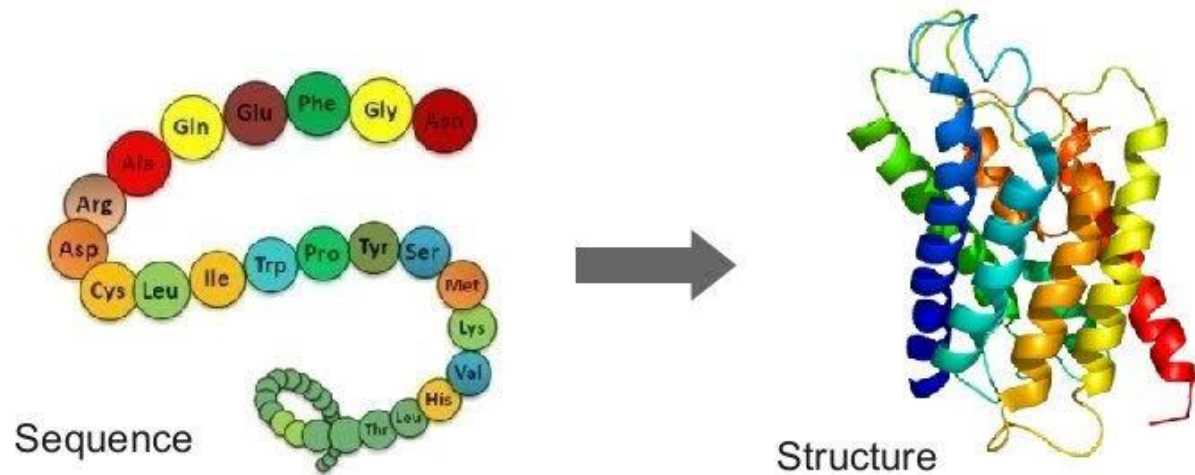
Our understanding of the physical world is fundamentally limited by our computational capabilities
(both in terms of *software* as well as *hardware*)

The interface between physics & computer science is generating lots of exciting developments (for *both* fields)
ex. quantum computing / quantum information

Computation spans across all area of physics

- **Biophysics:** structural prediction and function of biomolecules such as proteins and DNA
- **Condensed Matter Physics:**
 - electronic structure of solids and quantum effects
 - nonlinear and far from equilibrium processes
 - properties and dynamics of soft materials such as polymers, liquid crystals, colloids
- **Materials Physics:** behavior of complex materials, metals, alloys, composites
- **High Energy Physics:** lattice chromodynamics, theory of the strong interaction, data analysis from accelerator experiments
- **Astronomy and Cosmology:** formation and evolution of solar systems, star systems and galaxies
- **Fluids, Climate and Geophysics:** Navier–Stokes; weather and climate ensembles; seismic inversion; Earth's dynamo
- **Quantum Computing and Data Science:** quantum algorithms and simulation of quantum systems; machine learning on large experimental datasets; high-performance data analysis

Biophysics: protein structure prediction

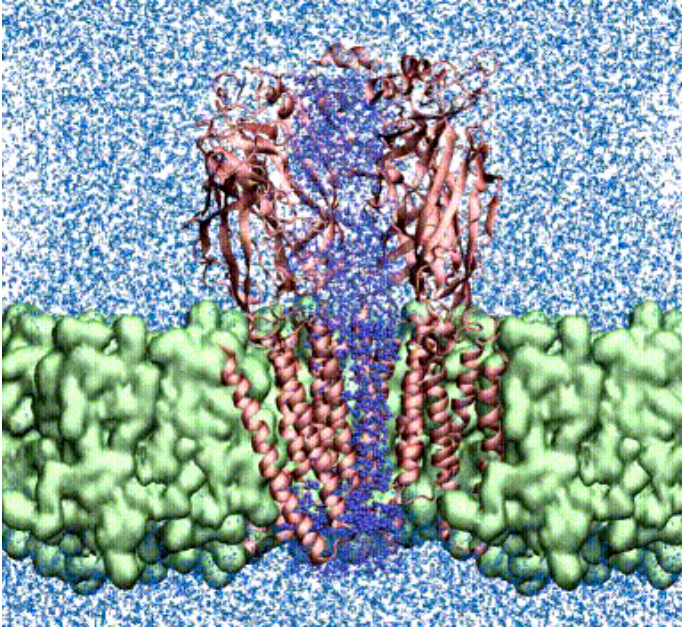


Proteins folds from sequence: sequence of amino acids which interact together to form complex three-dimensional structures (folds)

Hard problem because it is impossible to create an analytical model to predict the fold. The free-energy minimum is a delicate combination of enthalpic-entropic contributions

Need large-scale simulations (Anton supercomputer) or machine-learning models (see e.g. AlphaFold) based on previous knowledge and sequence similarities of other folds (obtained via x-ray or cryo-em).

Biophysics: biomolecules simulations



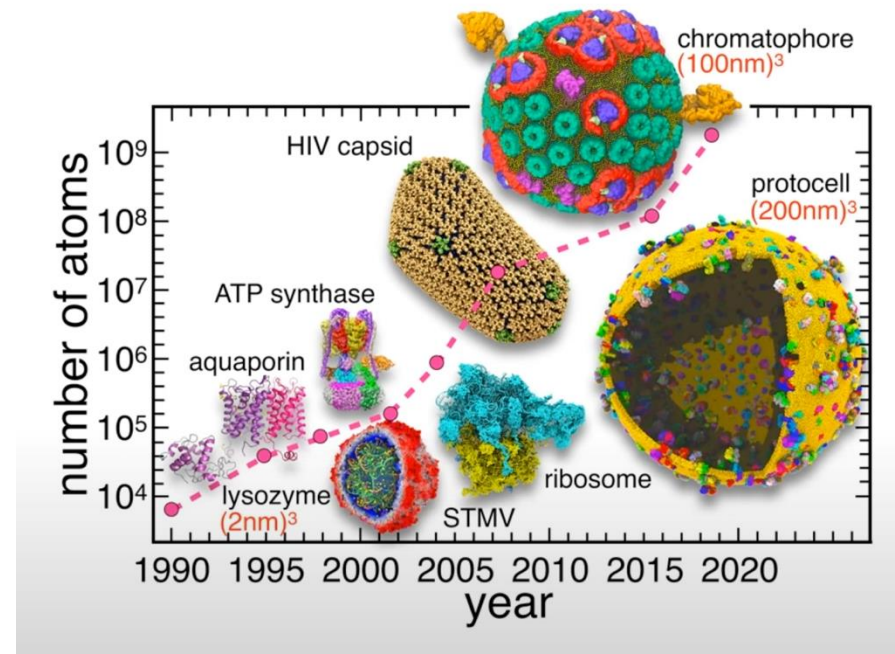
Now I have a collection of different bonded atoms, forming complex structures: how to setup a simulation?

Quantum or classical representation of atoms?

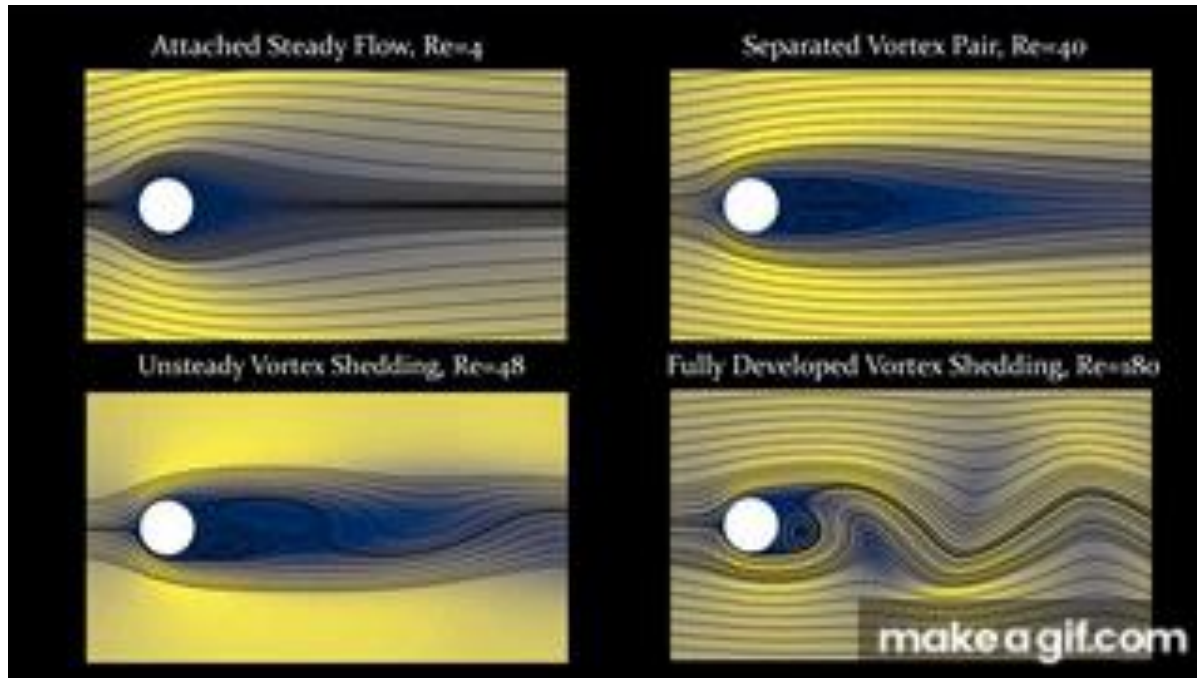
Changes a lot if I use Schoedinger's equation or Newton's one!

How to construct force field between atoms and boundary conditions?

We have atom organization of massive structures for which we want now dynamical information: require a large number of computational resources



Fluid dynamics: turbulence and the Navier–Stokes equations



Turbulence: many coupled scales (vortex shedding → Kolmogorov cascade)

Especially in turbulence conditions couples many length and time scales

Need large-scale simulations to get informations

Navier–Stokes: continuum equations for momentum and mass of a fluid. Goal: predict lift, drag, weather, mixing, and turbulence.

$$\partial_t \mathbf{u} + (\mathbf{u} \cdot \nabla) \mathbf{u} = -\frac{1}{\rho} \nabla p + \nu \nabla^2 \mathbf{u} + \mathbf{f}$$

$$\nabla \cdot \mathbf{u} = 0$$

The equations are nonlinear partial differential equations.

How to integrate them? Boundary conditions?

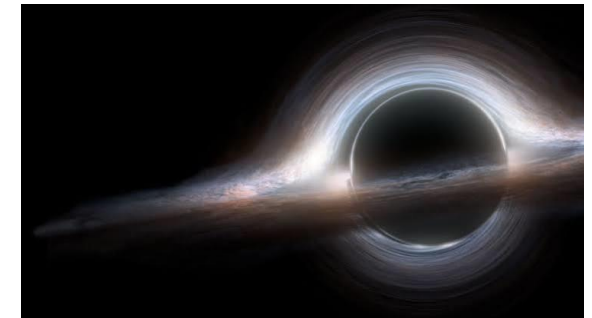
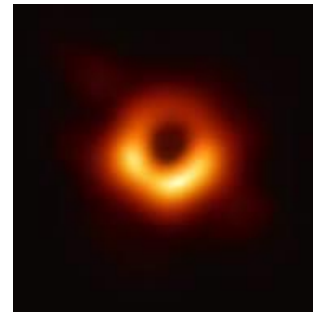
Astronomy: simulation of a black-hole



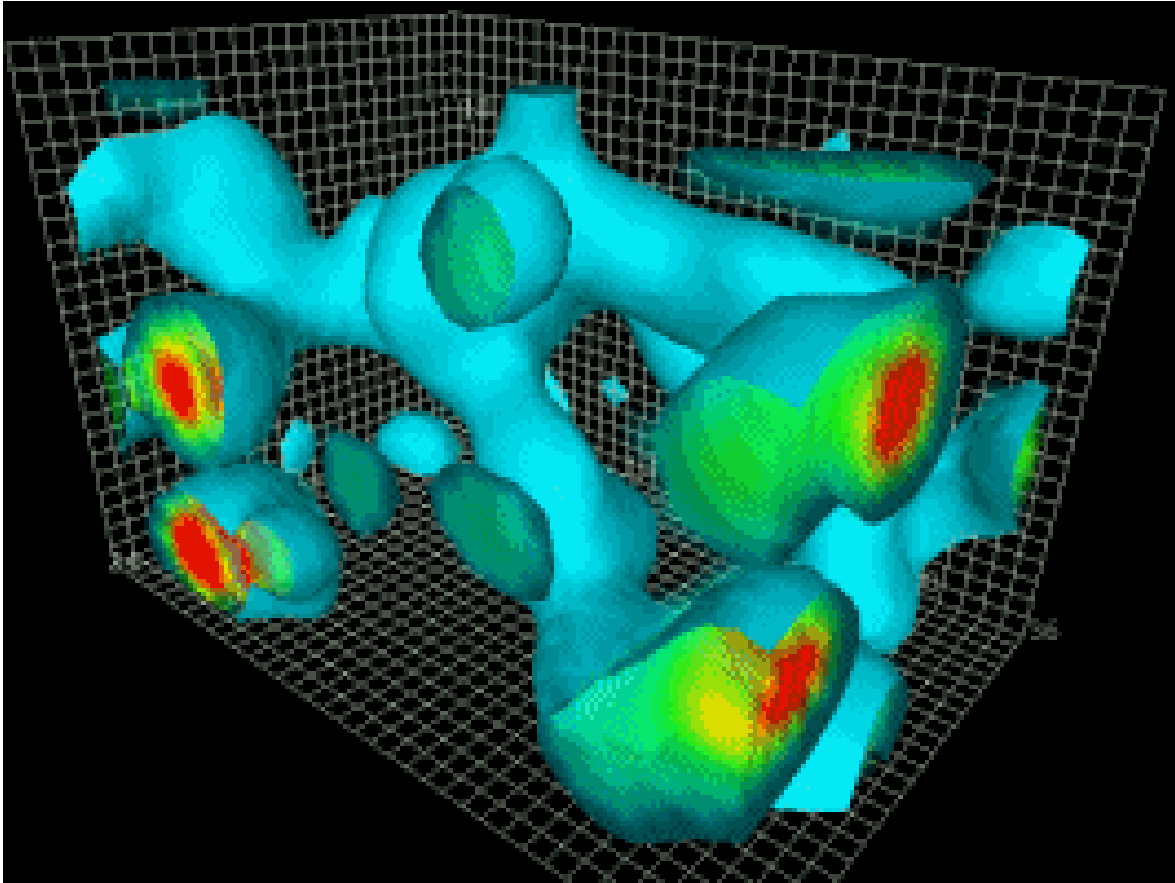
Einstein equations for spacetime plus ideal magnetohydrodynamics for the magnetized plasma (accretion disks, jets).

Coupled nonlinear PDEs on a dynamical curved background.

Need large-scale simulations on supercomputers.



High energy physics: lattice Quantum Chromodynamics

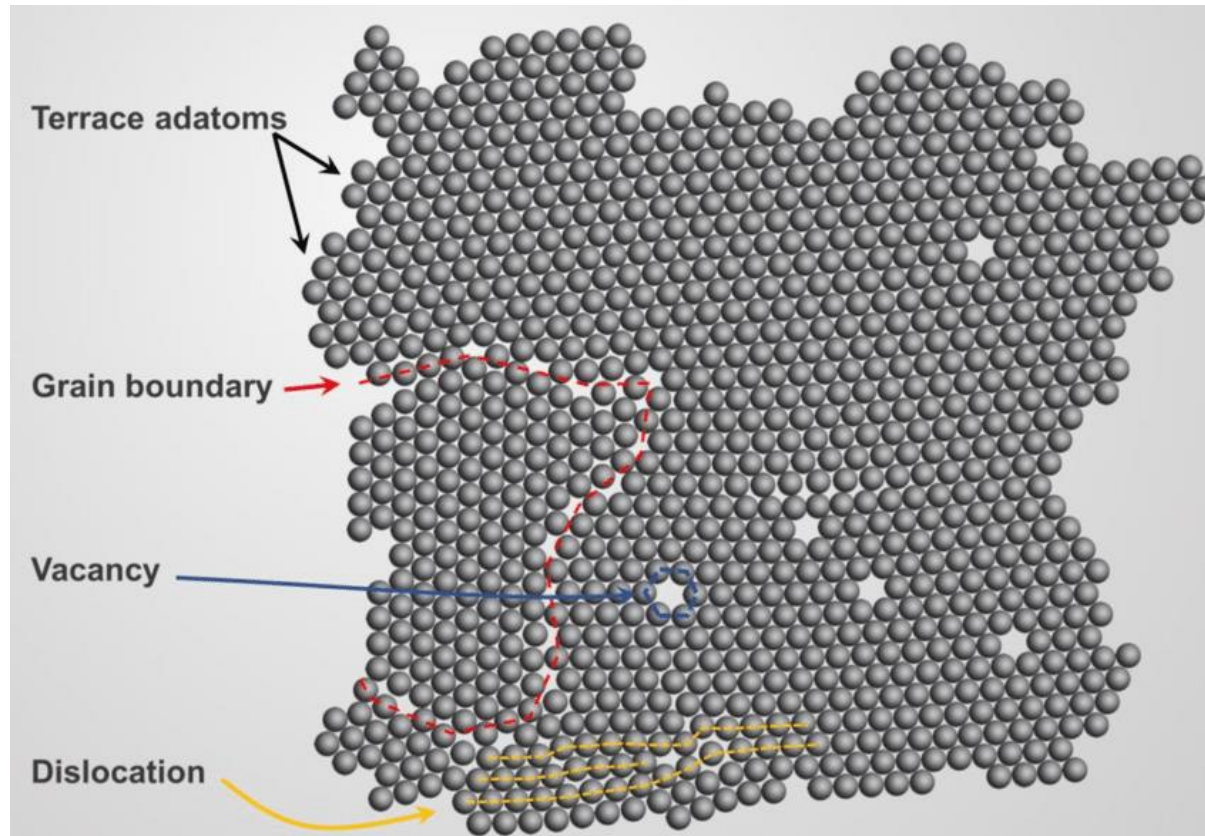


QCD: theory of the strong force. To measure e.g. hadron masses and decay constants

At low energy the coupling is large, so perturbation theory fails and quarks are confined.

Need large-scale lattice Monte Carlo simulations.

Materials physics: defects, strength, and multi-scale modeling



Defects: dislocations, grain boundaries, and vacancies control the strength, plasticity, and failure of metals, alloys, and composites.

Why it is hard?

The physics spans Ångströms and femtoseconds (electronic structure) up to millimeters and seconds (yield and fracture). No single model covers all scales.

Need density-functional theory, molecular dynamics, and continuum / dislocation-dynamics models, coupled across scales.

How to typically perform simulations

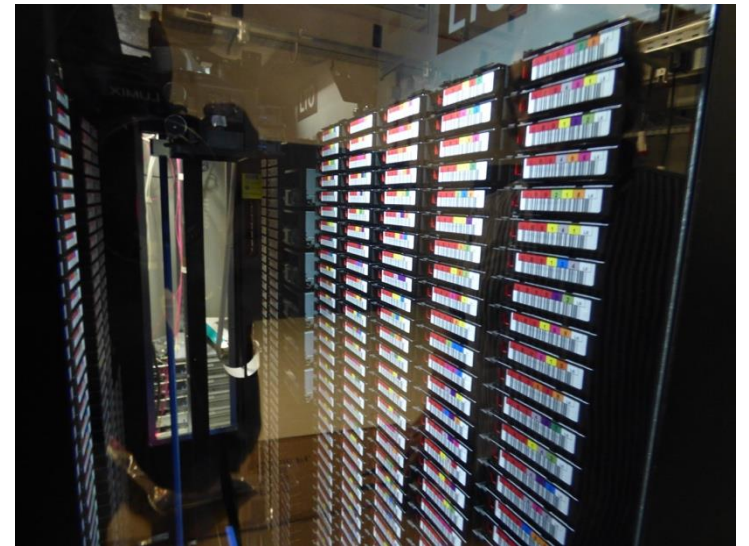
"Small simulations" on workstations such as this laptop

"Large simulations" on compute clusters or supercomputers may require lots of memory or calculation time distribute the problem over many (~ 10 to 100) processors either separately or "in parallel"



Leonardo (Cineca) in
Bologna
280000 cores calculation
nodes

RECAS in Bari
4000 cores



There are Italian and European grants that offer computational hours previous approval of scientific project

How a computational physicist works

- **Devise and implement** a computer model for the physical question of interest
- Needs **numerical mathematics** toolkit: discretization, error analysis, stability, efficiency
- Perform the computation
- **Analyse** and **visualize** the data
- Interpret and compare to experiment and theory
- Improve model predictions

How Large Language models will change forever the field of computational physics

LLMs can now do literature research, explain physical models, perform analytical calculations, write code — and explain it fluently.

Why this is a double-edged sword

You forget to reason yourself.

You stop thinking how to tackle the physical problem — and maybe miss a new method that is not in the model.

An LLM only contains knowledge accumulated up to now.

Suggestion: Avoid the temptation to immediately ask an LLM first where the error is. Try to find it yourself.

Programming Language for the course: Python

Strong features:

- Python has clear syntax
- fully object oriented
- huge body of implemented libraries (NumPy, SciPy, and Matplotlib)
- easy to learn
- free
- integration of fast compiled routines from other languages C/C++/Fortran
- ideal for teaching and writing algorithms: you can still read the code a week later.
- no compile step

Weak features:

- can be slow
- scripting language
- Python's package management can create portability problems

In Ch. 2 of reference book by Mark Newman you can review most basic constructs

Python syntax

Variables and arithmetic

```
a = 3
b = 2*a
print(a, b)
print(a+b, a-b, a*b, a**b, a/b, a//b, a%b)
```

Comparisons, and / or, if-else

```
print(a==b, a!=b)
print(a>b, a>=b)
print(a==3 and b==4)
print(a==3 or b==4)
if a > b:
    print("a is larger")
else:
    print("b is larger")
```

Python syntax

Importing libraries

```
import numpy as np
from math import sqrt
```

Lists

```
L = [1, 2, 3, 4]
L.append(5)
L.sort()
L.reverse()
```

NumPy arrays

```
a = np.array([0, 1, 2, 3, 4])
b = np.zeros(4, float)
c = np.zeros([3, 4], float)
d = np.linspace(0, 2, 5)
e = np.arange(0, 10, 2)
```

Python syntax

NumPy arrays

```
s = np.sum(a)
mean = np.mean(a)
var = np.var(a)
minimum = np.min(a)
maximum = np.max(a)
```

Access elements with `a[i]` in 1D and `a[i, j]` (or `a[i][j]`) in 2D. Access part of array with `a[start:end:stride]`
Remember that you can apply operations directly to the entire array. e.g. `np.exp(a)`

Loops

```
for i in range(5):
    print(i)
```

```
n = 0
while n < 5:
    print(n)
    n = n + 1
```

Defining a function

```
def f(x):
    return x**2 + 1

print(f(3))
```


Installing Python

Anaconda distribution <https://www.anaconda.com/>

Need to install if not present at least numpy, math, scipy, matplotlib packages
(go in Anaconda navigator -> environments)



Can program using:

- Python within Jupyter Notebook (examples will be given in this format)
- Pure Python (.py code) using e.g. **Visual Studio Code**
<https://code.visualstudio.com/>

The operating system is up to you, I will use MacOS.

Need a laptop where you can write, run code and plot.

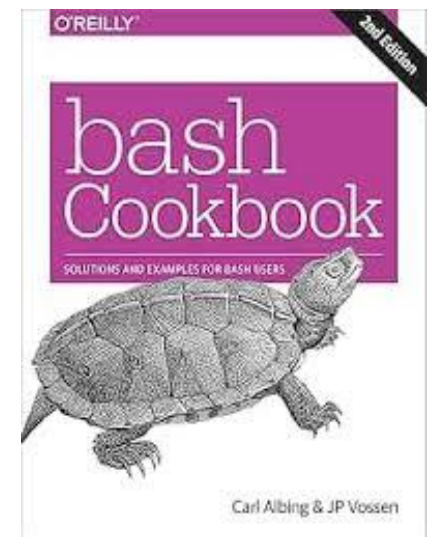
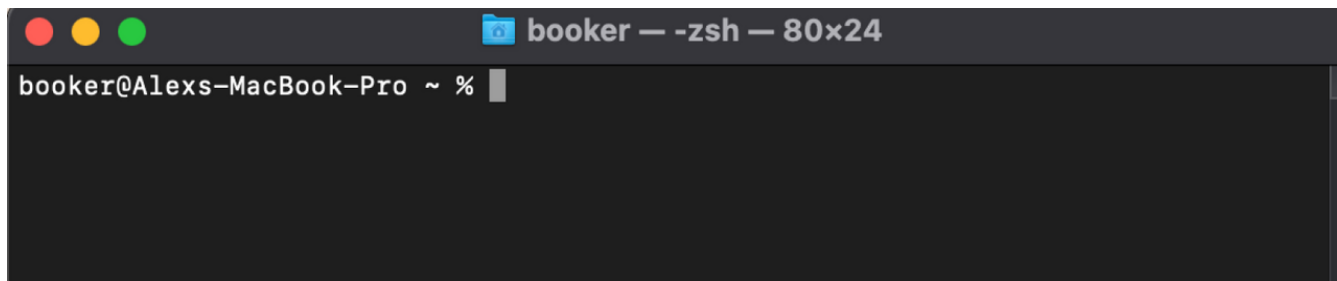
Programming Language for the course: bash

Strong features:

- Native on Linux and macOS
- Job scripts on clusters and supercomputers
- Pipes and text tools: grep, awk, cut, sed
- Glue language: launch Python, move files, chain commands
- Allow to manipulate massive amounts of data without fully opening them
- Perform basic analysis with awk
- No compile step

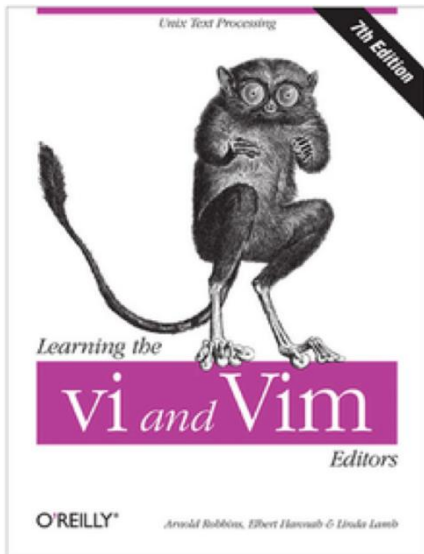
Weak features:

- Easy to write fragile scripts
- Quoting, spaces, and portability traps between different environments
- Not for heavy numerics — that is Python / C / Fortran
- Syntax is less readable than Python a week later

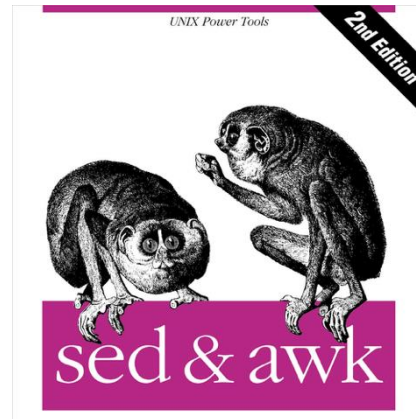


Programming Language for the course: bash

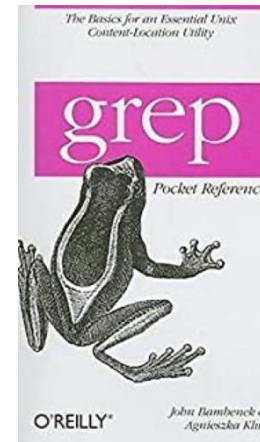
Actual useful functions:



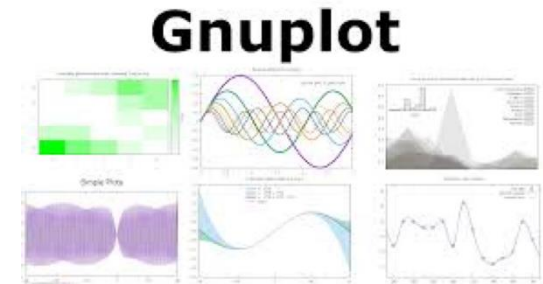
Editor



“substitute” and
“perform rapid
analysis”



Tool to “find”



Produce high-quality plots.
Alternative to matplotlib

Programming Language not in the course: C, C++, Fortran

Strong features:

- Portable and fast: compiled to machine code
- Large number of numerical libraries (BLAS, LAPACK, MPI)
- C++ is object-oriented; Fortran is still the core of many HPC physics codes
- Used inside packages you will only call from Python

Weak features:

- Steeper to learn than Python
- Easy to make silent mistakes: missing ';', wrong type, uninitialized variable
- Dynamic memory is error-prone (leaks, segfaults)
- Easy to write code that is fast in theory and slow in practice

When to use? If your code needs to run for months, and C/Fortran give a speed up factor of 10+, this makes a huge difference.

Testing your implementation can be challenging, so perhaps implement in Python and then port to C.

Python instead is perfect for fast calculations

Aims of the course

Provide you with a series of toolbox for solving physical problems on the computer

Understand how these toolbox are or can be implemented

Learn how to structure a project

For a given physical problem, you should be able to:

- identify suitable and efficient methods for solving it numerically
- write programs that use these algorithms
- sort out initial conditions and boundary conditions etc
- to test you program to make sure it is correct, especially think the type of tests to perform
- to read a code you didn't make, decipher it and read the flow
- to analyze the results
- have an overview of existing scientific numerical libraries

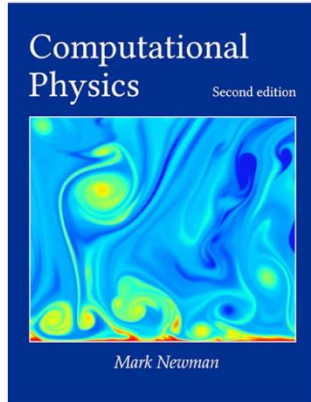
Course Syllabus

Description: Series of algorithms and numerical techniques to simulate classical and quantum mechanical problems on computers.

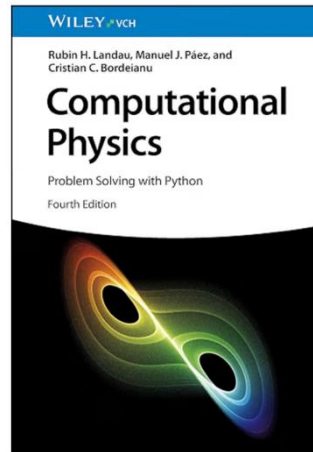
Topics:

- General introduction to scientific programming and visualization.
- Numerical integration and differentiation.
- Function interpolation.
- Linear algebra and matrices.
- Numerical solutions to (systems of) non-linear equations.
- Numerical solutions to ordinary and partial differential equations.
- Random processes and Monte Carlo methods.
- Important problems from classical, statistical, and quantum mechanics.

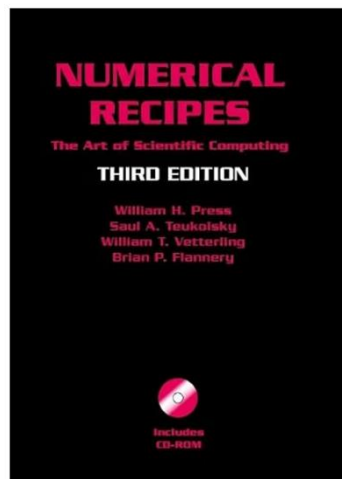
Textbooks



Computational Physics by Mark Newman (Some parts of this text plus the exercises are available on the author's website: <http://www-personal.umich.edu/~mejn/cp/index.html>)



Computational Physics: Problem Solving with Python by Rubin H. Landau et al.



Numerical Recipes 3rd Edition: The Art of Scientific Computing by William H. Press

Exam

- Oral exam with questions on the topics covered.
- A small computational project of your choice. Make a very small presentation and show the code written. Many exercises available in the book to choose from.